

Návrh a implementace databázových systémů

Uživatelské role

Procedurální rozšíření jazyka SQL

Uživatelské role

- PostgreSQL nabízí koncept uživatelských rolí
 - role může být
 - jediný uživatel (speciální případ)
 - skupina uživatelů
 - rolím lze udělovat práva
 - role mohou vlastnit objekty (tabulky apod.)
- vytvoření, modifikace, zrušení role

`CREATE ROLE name;`

`ALTER ROLE name RENAME TO newname;`

`DROP ROLE newname;`

Uživatelské role

- role může být definována s různými parametry, které specifikují práva role

- běžný uživatel

```
CREATE ROLE name LOGIN;  
CREATE USER name; -- totéž
```

- admin

- musím být admin, abych mohl tuto roli vytvořit

```
CREATE ROLE name SUPERUSER;
```

- možnost vytváření databáze

```
CREATE ROLE name CREATEDB;
```

- možnost vytváření dalších rolí

```
CREATE ROLE name CREATEROLE;
```

- práva role lze změnit pomocí **ALTER ROLE**

Uživatelské role

- uživatelé mohou být přiřazování do rolí, čímž jim mohou být hromadně udílána práva
 - funguje to tak, že se uživateli přiřadí role příkazem GRANT
 - vytvoříme „skupinovou“ roli
 - CREATE ROLE skupina CREATEDB;
 - přidáme člena
 - GRANT skupina TO jirka;
 - člen může být pochopitelně opět skupina

Uživatelské role

- aby člen skupiny měl automaticky i práva skupiny, musí být definován s atributem INHERIT
 - potom dědí práva každé skupiny, jejímž je členem
 - poněkud opačně, než by člověk čekal
 - dědičnost je definována zespoda, ne shora
- například

```
CREATE ROLE joe LOGIN INHERIT;  
CREATE ROLE admin NOINHERIT;  
CREATE ROLE wheel NOINHERIT;  
GRANT admin TO joe;  
GRANT wheel TO admin;
```

- joe dědí práva přímo nadřazených skupin, tj. skupiny admin, ale nedědí práva skupiny wheel, protože admin také nedědí práva skupiny wheel

Uživatelské role

- aby člen skupiny mohl užívat práva skupiny i když nemá INHERIT
- po přihlášení do databáze může uživatel použít příkaz **SET ROLE**
 - člen skupiny se může tímto příkazem „přepnout“ na práva kterékoli skupiny, jíž je členem
 - ztrácí tím ovšem práva, která má sám a skupina je nemá – operace není aditivní
 - nastavení přesně práv role admin
 - SET ROLE admin;**
 - nastavení původních práv
 - RESET ROLE;**

Procedurální rozšíření - motivace

- SQL je neprocedurální jazyk
 - říká CO chceme udělat, ne JAK
 - postrádá možnosti sofistikovanějšího řešení některých problémů
 - složitější integritní omezení
 - dotazy, které přirozeně vedou na cykly, podmínky a podobné programátorské konstrukce
 - mnohdy je obtížné přemýšlet „v množinách“, řešení nás často napadá snadněji s využitím klasických paradigmat sekvenčního funkcionálního programování
 - nevýhoda je také v tom, že pro čisté SQL (na serveru) a procedurální zpracování v klientovi se zvyšuje zátěž sítě

Procedurální rozšíření - motivace

- procedurální rozšíření SQL
 - doplnění SQL o možnost vestavěných funkcí, procedur, triggerů apod.
 - funkce lze volat například při událostech jako je vkládání dat, manipulace s daty a podobně
 - budeme používat volně dostupný SŘBD PostgreSQL, který disponuje procedurálním jazykem PL/pgSQL – obdoba PL/SQL firmy ORACLE

Typy funkcí v PostgreSQL

- SQL funkce
- procedurální funkce (psané v PL/pgSQL nebo jiném jazyce)
- vnitřní (interní funkce)
- funkce v jazyce C

- vstupní parametry
 - všechny základní i kompozitní datové typy
 - nebo nic
- návratová hodnota
 - jedna nebo více hodnot (set, množina) základního nebo kompozitního datového typu
 - void

SQL funkce

- zapouzdřuje běžné SQL dotazy
- vrací výsledek posledního SQL příkazu
 - mohou vracet pouze první řádek nebo celou výslednou tabulku
 - nemusí také vracet nic, pokud je tak deklarováno
 - RETURNS SETOF integer
 - RETURNS TABLE(columns)
 - RETURNS void
- může obsahovat SELECT bloky, INSERT, UPDATE, DELETE, ...
 - pokud není poslední příkaz SELECT a funkce má něco vracet, musí se použít RETURNING

SQL funkce

- nemůže obsahovat řízení transakcí
- přístup k parametrům
 - číslování, např. první parametr je **\$1**
 - pokud jsou parametry strukturované, použijeme tečkovou notaci, např. **\$3.name**
 - parametry mohou být použity pouze jako datové hodnoty, ne jako identifikátory
 - lze používat výstupní parametry:

```
CREATE FUNCTION soucet(integer, integer, OUT sum integer) AS  
$$  
    SELECT $1+$2;  
$$ LANGUAGE SQL;
```

SQL funkce

- deklarace

```
CREATE FUNCTION jméno(parlist) RETURNS datatype AS  
$$
```

```
    příkazy...
```

```
$$ LANGUAGE SQL;
```

- tělo funkce musí být deklarováno jako řetězec, tj. uzavřeme jej například do \$\$

- volání: **SELECT fname();**

- volání funkce lze použít i v klauzuli FROM

- odstranění: **DROP FUNCTION jméno(parlist);**

SQL funkce

■ příklady

```
CREATE FUNCTION clean_emp() RETURNS void AS $$  
    DELETE FROM emp WHERE salary < 0;  
$$ LANGUAGE SQL;
```

```
AS  
CREATE FUNCTION add(integer, integer) RETURNS integer  
$$  
    SELECT $1+$2;  
$$ LANGUAGE SQL;
```

```
SELECT add(1,2) AS answer;  
DROP add(int,int);
```

SQL funkce

■ příklady

```
CREATE FUNCTION tf1(integer, numeric) RETURNS integer
AS
$$
    UPDATE bank
        SET balance = balance-$2
        WHERE accountNo = $1
    RETURNING balance;
$$ LANGUAGE SQL;

SELECT tf1(17, 100.0);
```

SQL funkce

- použití kompozitních typů

```
CREATE TABLE emp (name text, salary numeric, age integer);
```

```
CREATE FUNCTION doubleSalary(emp) RETURNS numeric AS $$  
    SELECT $1.salary * 2 AS salary;  
$$ LANGUAGE SQL;
```

```
SELECT name, doubleSalary(emp.*)  
FROM emp  
WHERE name='Jenda';
```

SQL funkce

- použití kompozitních typů
 - pokud je použit kompozitní typ jako návratová hodnota, vrací se jen jeden řádek!

```
CREATE FUNCTION newEmp() RETURNS emp AS $$  
    SELECT text 'None' AS name,  
           1000.0 AS salary,  
           25 AS age;  
$$ LANGUAGE SQL;
```

```
SELECT (new_emp()).name;
```

SQL funkce

- použití kompozitních typů
 - jak vrátit víc řádků

```
CREATE FUNCTION getOldEmp(integer) RETURNS SETOF emp  
AS $$
```

```
    SELECT * FROM emp WHERE age>$1;  
$$ LANGUAGE SQL;
```

```
CREATE FUNCTION sum_n_product (x int)  
RETURNS TABLE(sum int, product int) AS $$
```

```
    SELECT $1 + tab.y,  $1 * tab.y  
    FROM tab;  
$$ LANGUAGE SQL;
```